

Nucleo de Situacao de Saude – UENF

Front-end: Arquitetura V1, Navegacao Geografica e Escopo de Demonstracao

Documentacao para implementacao incremental

15 de setembro de 2026

Decisao arquitetural

Esta V1 prioriza uma demonstracao visual forte e funcional, mantendo a arquitetura definitiva do NSS. O mapa deixa de ser um elemento opcional e passa a ser a principal forma de navegacao da interface. O front-end conversa somente com a API Java quando estiver em modo real; enquanto os endpoints epidemiologicos ainda nao estiverem disponiveis, a mesma interface opera com dados mockados pelo mesmo contrato tipado.

Sumário

1	Objetivo desta V1	4
2	Arquitetura do NSS e limite do front-end	4
3	Estado atual do componente Java/Spring Boot	5
3.1	O que já existe	5
3.2	O que ainda falta para o papel arquitetural definitivo	5
4	Contrato de dados PostgreSQL para a V1	5
5	Contrato mínimo da API Java para o front-end	6
5.1	Health check	6
5.2	Doenças disponíveis	6
5.3	Agregado municipal	7
6	Arquitetura do front-end	7
6.1	Stack	7
6.2	Estrutura sugerida	7
6.3	Abstração de fonte de dados	8
7	Arquitetura cartográfica da V1	9
7.1	Decisão: GeoJSON vetorial em vez de PNGs em grid	9
7.2	Arquivos cartográficos necessários	9
7.3	Script de geração	9
8	Modelo de navegação geográfica	10
8.1	Nível 1: Brasil por regiões	10
8.2	Nível 2: estados do Sudeste	10
8.3	Nível 3: municípios do Rio de Janeiro	10
8.4	Breadcrumb e retorno	11
9	Estado de navegação e estado epidemiológico	11
9.1	Nível do mapa	11
9.2	Geografia selecionada	11
9.3	Filtros epidemiológicos	11
10	Join entre geometria e dados epidemiológicos	12
11	Cobertura e semântica de dados	12
11.1	Diferenciar zero de ausência de dados	12
11.2	Agregações incompletas	12
11.3	Metadados de demonstração	13
12	Especificação visual e responsividade	13
12.1	Princípio mobile-first	13
12.2	Mobile	13
12.3	Telas médias e grandes	14
12.4	Conteúdo do painel	14

13 Interacao: mouse, toque e teclado	14
14 Visualizacao epidemiologica no mapa	15
15 Zoom e transicoes	15
16 Tela unica da V1	15
17 Mock da demonstracao	16
18 Estados obrigatorios da interface	16
19 Escopo explicitamente fora da V1	16
20 Ordem de implementacao recomendada ao Codex	17
21 Criterios de aceite da V1	18
22 Evolucao posterior	18
23 Resumo executivo	18

1 Objetivo desta V1

Este documento define a primeira versao arquitetural do repositorio `nss-front-end`. O objetivo imediato e permitir uma demonstracao apresentavel em curto prazo, com escopo controlado, responsividade real e navegacao geografica progressiva, sem criar uma implementacao descartavel.

A V1 deve entregar uma unica dashboard epidemiologica contendo:

- cabeçalho institucional do NSS/UENF;
- mapa vetorial interativo como elemento central;
- navegacao geografica progressiva Brasil → Sudeste → Rio de Janeiro;
- filtros por doenca, ano e mes;
- painel contextual com selecao atual, total do recorte quando semanticamente valido e legenda;
- ranking dos municipios com dados no nivel do Rio de Janeiro;
- estados de carregamento, erro, ausencia de dados e ausencia de cobertura;
- suporte a mock e API real por uma mesma abstracao de acesso a dados;
- comportamento mobile-first e adaptacao para telas maiores.

Nao fazem parte desta primeira entrega: autenticacao visual, administracao de usuarios, dashboards multiplas, filtros demograficos, cartografia de ruas, geocodificacao, GPS, WebSocket, SSR, exportacoes ou um sistema de design complexo.

2 Arquitetura do NSS e limite do front-end

A arquitetura de referencia do NSS separa quatro componentes:

1. **Front-end:** React + TypeScript, responsavel pela interface e visualizacoes;
2. **Java / Spring Boot:** API principal, autenticacao, autorizacao, regras de negocio e leitura do PostgreSQL;
3. **PostgreSQL:** persistencia dos dados consolidados e dados da aplicacao;
4. **Python / PySUS:** pipeline de ingestao e transformacao do SINAN, fora do caminho sincrono de consulta do usuario.

Decisao arquitetural

O front-end nao acessa PostgreSQL e nao chama o pipeline Python/PySUS. Toda consulta epidemiologica da aplicacao deve passar pela API Java. A geometria cartografica local em GeoJSON e um recurso estatico do front-end e nao representa acesso ao pipeline epidemiologico.

O fluxo desejado e:

```
Usuario -> Front-end React -> API Java -> PostgreSQL
      ^
```

3 Estado atual do componente Java/Spring Boot

O projeto Java analisado já possui elementos importantes para ocupar o papel de backend principal, mas ainda representa apenas o início dessa responsabilidade.

3.1 O que já existe

A base atual utiliza Java 21, Spring Boot 4.1.1, Spring Web MVC, Spring Data JPA, Bean Validation, PostgreSQL, Flyway, Spring Security, DTOs, MapStruct, Lombok, Dockerfile e Docker Compose.

A entidade atualmente implementada é `User`, com CRUD HTTP em `/NSS/users`. Ainda não existem os recursos epidemiológicos que o front-end desta V1 deverá consumir.

3.2 O que ainda falta para o papel arquitetural definitivo

Ainda precisam ser implementados no backend Java:

- entidades e repositórios para agregados epidemiológicos;
- endpoints para doenças e agregados municipais;
- migrations Flyway para o schema;
- contrato padronizado de erros;
- configuração CORS para o front-end;
- especificação OpenAPI;
- testes de integração do banco e da API epidemiológica;
- autenticação e autorização definitivas em fase posterior.

Atenção

O arquivo `src/main/resources/application.yml` do projeto Java precisa ser revisado: no estado analisado, ele contém definições equivalentes a Docker Compose em vez de propriedades Spring. A definição dos serviços pertence ao `compose.yml`; o `application.yml` deve conter configurações da aplicação.

4 Contrato de dados PostgreSQL para a V1

A tabela epidemiológica proposta para a primeira integração é deliberadamente pequena e espelha o Gold mensal que o pipeline atual já produz.

```
CREATE TABLE sinan_case_aggregate (  
  id BIGSERIAL PRIMARY KEY,  
  disease VARCHAR(20) NOT NULL,  
  source_year SMALLINT NOT NULL,
```

```
year SMALLINT NOT NULL CHECK (year >= 2000),
month SMALLINT NOT NULL CHECK (month BETWEEN 1 AND 12),
cd_uf CHAR(2) NOT NULL,
nm_uf VARCHAR(100) NOT NULL,
cd_mun CHAR(7) NOT NULL,
nm_mun VARCHAR(150) NOT NULL,
cases_total INTEGER NOT NULL CHECK (cases_total >= 0),
batch_id VARCHAR(32) NOT NULL,
ingested_at TIMESTAMPTZ NOT NULL,

CONSTRAINT uq_sinan_case_aggregate_v1 UNIQUE
    (disease, source_year, year, month, cd_uf, cd_mun)
);

CREATE INDEX idx_sinan_case_filter
ON sinan_case_aggregate (disease, year, month, cd_uf);
```

Decisao arquitetural

A V1 utiliza apenas casos mensais agregados por municipio. Sexo, classificacao final, evolucao, semana epidemiologica, idade e outras dimensoes ficam fora desta primeira demonstracao.

A cobertura de dados de demonstracao fica limitada a quatro municipios do Rio de Janeiro: Campos dos Goytacazes, Sao Joao da Barra, Itaperuna e Macae.

5 Contrato minimo da API Java para o front-end

A API V1 deve usar prefixo versionado /api/v1.

5.1 Health check

```
GET /api/v1/health
```

```
200 OK
{
  "status": "ok"
}
```

5.2 Doencas disponiveis

```
GET /api/v1/diseases
```

```
200 OK
{
  "items": ["DENG"]
}
```

5.3 Agregado municipal

```
GET /api/v1/epidemiology/municipalities?disease=DENG&year=2026&month=1

200 OK
{
  "disease": "DENG",
  "year": 2026,
  "month": 1,
  "items": [
    {
      "cdUf": "33",
      "nmUf": "Rio de Janeiro",
      "cdMun": "3301009",
      "nmMun": "Campos dos Goytacazes",
      "casesTotal": 123
    }
  ]
}
```

Esse endpoint permite ao front-end calcular ranking municipal, total do recorte municipal disponível e intensidade visual dos municípios cobertos.

6 Arquitetura do front-end

6.1 Stack

Para a V1, utilizar:

- React;
- TypeScript com `strict` habilitado;
- Vite;
- Tailwind CSS;
- TanStack Query para estado de servidor;
- Fetch API nativa como cliente HTTP;
- React Simple Maps para renderizacao SVG das geometrias GeoJSON.

Evitar Redux, Zustand, Next.js, bibliotecas de formulario, bibliotecas de mapas baseadas em tiles e demais dependencias sem necessidade demonstrada.

6.2 Estrutura sugerida

```
src/
|-- app/
| |-- App.tsx
| '-- providers.tsx
|-- api/
| |-- http.ts
| '-- epidemiology.ts
```

```
|-- features/  
| '-- dashboard/  
| |-- DashboardPage.tsx  
| |-- components/  
| | |-- GeographicMap/  
| | | |-- GeographicMap.tsx  
| | | |-- RegionLayer.tsx  
| | | |-- StateLayer.tsx  
| | | |-- MunicipalityLayer.tsx  
| | | |-- MapTooltip.tsx  
| | | '-- MapLegend.tsx  
| | |-- GeographyBreadcrumb.tsx  
| | |-- FilterPanel.tsx  
| | |-- FilterDrawer.tsx  
| | '-- MunicipalityRanking.tsx  
| '-- hooks/  
| |-- useMapNavigation.ts  
| '-- useEpidemiologyQuery.ts  
|-- data/  
| |-- dataSource.ts  
| |-- apiDataSource.ts  
| '-- mockDataSource.ts  
|-- mocks/  
| '-- epidemiology.ts  
|-- types/  
| '-- epidemiology.ts  
|-- index.css  
'-- main.tsx
```

6.3 Abstracao de fonte de dados

A interface visual nao deve saber se os dados vieram do mock ou do Java.

```
interface EpidemiologyDataSource {  
  listDiseases(): Promise<string[]>;  
  getMunicipalityCases(filters: {  
    disease: string;  
    year: number;  
    month: number;  
  }): Promise<MunicipalityCasesResponse>;  
}
```

O modo e escolhido por variavel de ambiente:

```
VITE_API_BASE_URL=http://localhost:8080  
VITE_USE MOCKS=true
```

A troca para a API real nao deve exigir alteracao nos componentes visuais.

7 Arquitetura cartografica da V1

7.1 Decisao: GeoJSON vetorial em vez de PNGs em grid

A cartografia deve ser gerada previamente por Python usando `geobr`/GeoPandas, mas persistida no front-end como GeoJSON simplificado. Nao utilizar imagens PNG individuais remontadas em grid como base da interacao.

Motivos:

- SVG/GeoJSON preserva as geometrias e permite hitboxes reais;
- hover, focus, clique e estilos ficam associados ao poligono correto;
- a responsividade nao depende de alinhar imagens rasterizadas;
- cores de heatmap podem ser aplicadas dinamicamente;
- a mesma geometria funciona em diferentes resolucoes de tela;
- acessibilidade por teclado e mais simples de implementar;
- nao e necessario gerar milhares de imagens de municipios.

7.2 Arquivos cartograficos necessarios

A V1 precisa somente destes recursos estaticos:

```
public/maps/  
|-- brazil-regions.geojson  
|-- southeast-states.geojson  
'-- rj-municipalities.geojson
```

O primeiro deve conter as cinco regioes brasileiras; o segundo, somente ES, MG, RJ e SP; o terceiro, os municipios do estado do Rio de Janeiro.

7.3 Script de geracao

Criar um script de desenvolvimento, por exemplo `scripts/generate_maps.py`, responsavel por:

1. ler as regioes com `geobr.read_region`;
2. ler os estados e filtrar o Sudeste;
3. ler os municipios do RJ com `geobr.read_municipality(code_muni="RJ")`;
4. manter os codigos oficiais necessarios em `properties`;
5. simplificar a geometria somente se necessario para reduzir tamanho, sem deformacao perceptivel;
6. exportar GeoJSON valido em `public/maps`;
7. nao depender de Python em runtime no navegador.

Decisao arquitetural

O Python desta secao e apenas uma ferramenta de build cartografico. Ele nao substitui o pipeline PySUS e nao entra no caminho de consulta da aplicacao.

8 Modelo de navegacao geografica

O mapa deve funcionar como uma navegacao hierarquica de tres niveis na V1:

```
BRASIL
|- Norte
|- Nordeste
|- Centro-Oeste
|- Sul
'- Sudeste -----> SUDESTE
    |- ES
    |- MG
    |- SP
    '- RJ -----> RIO DE JANEIRO
        |- Campos dos Goytacazes
        |- Sao Joao da Barra
        |- Itaperuna
        '- Macae
```

8.1 Nivel 1: Brasil por regioes

No carregamento inicial, exibir as cinco regioes brasileiras.

Todas devem possuir hover, focus e selecao visual. Entretanto, somente **Sudeste** permite drill-down na V1. As demais regioes podem ser selecionadas para mostrar nome e estado de cobertura, mas nao devem navegar para outro nivel.

Nao apresentar 0 **casos** para regioes sem cobertura. Utilizar texto como **Dados ainda nao disponiveis** nesta V1.

8.2 Nivel 2: estados do Sudeste

Ao ativar Sudeste, o mapa deve mudar para ES, MG, RJ e SP. Todos os estados devem ser identificaveis por hover/focus/toque.

Somente o **Rio de Janeiro** permite drill-down na V1. SP, MG e ES devem informar ausencia de cobertura desta demonstracao, sem inventar total zero.

8.3 Nivel 3: municipios do Rio de Janeiro

Ao ativar RJ, renderizar a malha municipal do estado para preservar a forma geografica completa.

Todos os municipios podem aparecer graficamente, mas somente quatro possuem dados interativos epidemiologicos nesta V1:

- Campos dos Goytacazes;

- Sao Joao da Barra;
- Itaperuna;
- Macae.

Os municipios sem cobertura devem permanecer visualmente neutros e nao podem ser interpretados como municipios com zero casos.

8.4 Breadcrumb e retorno

Exibir breadcrumb do nivel atual:

```
Brasil  
Brasil > Sudeste  
Brasil > Sudeste > Rio de Janeiro
```

Os niveis anteriores devem ser clicaveis. Tambem deve existir uma acao clara **Voltar** em contexto apropriado.

9 Estado de navegacao e estado epidemiologico

Separar explicitamente tres conceitos no codigo.

9.1 Nivel do mapa

```
type MapLevel =  
  | "BRAZIL_REGIONS"  
  | "SOUTHEAST_STATES"  
  | "RJ_MUNICIPALITIES";
```

9.2 Geografia selecionada

```
type GeographySelection = {  
  level: "region" | "state" | "municipality";  
  code: string;  
  name: string;  
};
```

9.3 Filtros epidemiologicos

```
type EpidemiologyFilters = {  
  disease: string;  
  year: number;  
  month: number;  
};
```

Decisao arquitetural

`mapLevel`, `selectedGeography` e `epidemiologyFilters` nao devem ser fundidos em um unico estado monolitico. Essa separacao reduz acoplamento e simplifica a evolucao da navegacao.

10 Join entre geometria e dados epidemiologicos

GeoJSON responde *onde* a unidade geografica esta e qual seu codigo. A API responde *quantos casos* existem no filtro selecionado.

A ligacao deve usar codigo oficial, nunca nome textual como chave principal.

```
GeoJSON feature
  properties.code_muni = "3301009"
    |
    v
API item
  cdMun = "3301009"
  casesTotal = 120
```

A geometria nao deve armazenar valores epidemiologicos fixos.

11 Cobertura e semantica de dados

11.1 Diferenciar zero de ausencia de dados

A interface deve distinguir visual e semanticamente:

- **sem cobertura**: nao ha dados disponiveis nesta V1;
- **zero casos**: existe cobertura para a unidade e o total calculado e zero;
- **valor positivo**: existe cobertura e ha casos registrados no recorte.

Cinza neutro deve representar **sem cobertura**; a menor intensidade da escala epidemiologica deve representar zero ou baixa incidencia conforme a regra visual escolhida.

11.2 Agregacoes incompletas

Atencao

O front-end nao deve somar os quatro municipios cobertos e apresentar esse valor como total do estado do Rio de Janeiro, da regioao Sudeste ou do Brasil. Esses seriam totais incompletos e semanticamente incorretos.

Na V1, niveis sem cobertura completa servem primordialmente como navegacao. Totais agregados so podem ser apresentados quando o backend declarar cobertura suficiente para a unidade geografica.

11.3 Metadados de demonstracao

O mock pode declarar capacidades de drill-down separadamente da disponibilidade de dados:

```
const demoCoverage = {
  drilldownEnabled: {
    regions: ["SE"],
    states: ["RJ"],
  },
  caseDataAvailableForMunicipalities: [
    "3301009",
    "3305000",
    "3302205",
    "3302403",
  ],
};
```

12 Especificacao visual e responsividade

12.1 Principio mobile-first

A interface deve ser implementada primeiro para viewport estreita e depois expandida progressivamente. Nao criar um desktop fixo para depois apenas empilhar elementos no celular.

12.2 Mobile

Em telas pequenas:

- o mapa ocupa **100% da largura disponivel**;
- o painel de filtros e informacoes fica oculto por padrao;
- uma acao visivel abre o painel como Drawer/Sheet lateral ou sobreposto;
- o Drawer deve fechar por botao, toque fora e tecla **Escape**;
- ao abrir, o foco deve ir para o painel e permanecer controlado enquanto estiver ativo;
- nao depender de hover para nenhuma informacao essencial;
- toque em uma geografia deve equivaler a clique/selecionar.

Representacao conceitual:

```
+-----+
| NSS / UENF |
+-----+
| breadcrumb |
+-----+
| |
| MAPA |
| 100% |
| |
+-----+
```

```
| [Filtros e informacoes] |
+-----+
```

12.3 Telas medias e grandes

Quando houver largura suficiente, trocar para duas colunas:

```
+-----+-----+
| | |
| MAPA | Filtros |
| ~ 70% | Selecao atual |
| | Informacoes |
| | Legenda |
| | |
+-----+-----+
```

A proporcao inicial deve ser aproximadamente 70/30, mas o breakpoint e a distribuicao podem ser ajustados apos teste visual. Nao assumir obrigatoriamente `md`; testar `md` e `lg` e usar o primeiro ponto em que a leitura do painel e do mapa permaneça confortável.

O painel lateral deve preferir `position: sticky` dentro de CSS Grid/Flex, e nao `position: fixed`, para permanecer integrado ao fluxo da pagina.

12.4 Conteudo do painel

O painel deve conter, no maximo:

1. filtro de doenca;
2. filtro de ano;
3. filtro de mes;
4. nome da geografia selecionada;
5. informacao de cobertura;
6. total apenas quando semanticamente valido;
7. legenda do mapa;
8. indicador discreto **DEMO** quando estiver usando mock.

13 Interacao: mouse, toque e teclado

Toda geografia interativa deve ter comportamentos equivalentes para diferentes dispositivos.

Entrada	Comportamento esperado
Mouse hover	realçar polígono e mostrar nome/informação curta
Focus por teclado	mesmo destaque essencial do hover
Clique	selecionar geografia e, quando habilitado, fazer drill-down
Toque	selecionar; se a unidade permitir drill-down, executar navegação de forma previsível
Enter/Space	equivalente à ativação por clique para geografia focada
Escape	fechar Drawer/Sheet quando aberto

Não esconder informação indispensável exclusivamente em tooltip de hover.

14 Visualização epidemiológica no mapa

A intensidade de cor deve ser derivada de `casesTotal`. A V1 pode utilizar uma escala discreta simples, desde que a legenda explique seu significado.

Regras obrigatórias:

- município sem cobertura: cinza neutro;
- município com cobertura e zero casos: estilo válido distinto de sem cobertura;
- município com valor positivo: intensidade crescente;
- estado/região sem cobertura completa: não receber cor que sugira total epidemiológico completo;
- contraste suficiente entre estados normal, hover/focus e selecionado.

Não é necessário implementar uma escala epidemiológica sofisticada ou normalizada por população nesta V1.

15 Zoom e transições

A V1 não precisa calcular zoom GIS sofisticado nem animar bounds dinamicamente. Podem ser utilizados presets de visualização para os três níveis.

```
const MAP_VIEWS = {
  brazil: { center: [-54, -15], zoom: 1 },
  southeast: { center: [-45, -20], zoom: 2.2 },
  rioDeJaneiro: { center: [-42.5, -22.2], zoom: 5 },
};
```

Os valores finais devem ser ajustados visualmente. Transições curtas podem ser usadas, mas não são critério de aceite se colocarem o prazo em risco.

16 Tela única da V1

A página deve seguir esta hierarquia visual:

1. Header institucional;

2. breadcrumb geografico;
3. area principal contendo mapa + painel responsivo;
4. ranking municipal abaixo, exibido quando fizer sentido no nivel do RJ;
5. estados de loading, error, empty e sem cobertura integrados ao fluxo.

Os antigos cards KPI no topo deixam de ser obrigatorios. Informacoes resumidas devem preferencialmente aparecer no painel contextual para preservar o mapa como elemento principal.

17 Mock da demonstracao

O mock deve usar o mesmo formato da futura resposta Java.

```
DENG / 2026 / janeiro
- Campos dos Goytacazes: 120
- Macae: 85
- Itaperuna: 42
- Sao Joao da Barra: 18
```

Os valores sao sinteticos e devem ser identificados como demonstracao. Nunca apresenta-los como estatistica epidemiologica real.

18 Estados obrigatorios da interface

Toda consulta epidemiologica deve tratar:

- **loading**: skeleton/placeholder sem deslocamento importante de layout;
- **error**: mensagem curta e acao de tentar novamente;
- **empty**: existe cobertura, mas nao ha registros para o filtro;
- **no coverage**: a V1 nao possui cobertura para a unidade geografica;
- **success**: renderizacao normal de dados.

19 Escopo explicitamente fora da V1

Para proteger a entrega, nao implementar agora:

- login, cadastro, recuperacao de senha ou RBAC;
- tela CRUD de usuarios;
- consumo direto do FastAPI antigo;
- acesso direto ao PostgreSQL;
- filtros por sexo, classificacao, evolucao, idade ou semana epidemiologica;
- todos os municipios do Brasil como areas epidemiologicas ativas;
- drill-down fora de Sudeste e RJ;

- mapas de ruas, satellite ou tiles;
- exportacao PDF/CSV;
- notificacoes em tempo real;
- WebSocket;
- SSR/Next.js;
- dashboard builder;
- cobertura de testes extensa;
- animacao cartografica sofisticada.

20 Ordem de implementacao recomendada ao Codex

1. inicializar Vite + React + TypeScript;
2. configurar Tailwind e TanStack Query;
3. criar tipos epidemiologicos e estados de navegacao geografica;
4. criar `EpidemiologyDataSource`;
5. implementar `mockDataSource`;
6. criar `scripts/generate_maps.py`;
7. gerar e versionar os tres GeoJSONs da V1;
8. implementar o nivel Brasil/regioes;
9. implementar drill-down do Sudeste;
10. implementar drill-down do RJ com malha municipal;
11. fazer join por codigo IBGE entre GeoJSON e mock;
12. implementar breadcrumb e navegacao de retorno;
13. implementar painel lateral desktop e Drawer mobile reutilizando o mesmo conteudo de filtros;
14. implementar legenda e estados sem cobertura;
15. implementar ranking dos quatro municipios;
16. implementar `apiDataSource`;
17. validar responsividade, teclado, toque e mouse;
18. executar `npm run build` e `npm run lint`.

21 Critérios de aceite da V1

Critério	Aceite
Build	npm run build termina com sucesso
Lint	sem erros bloqueadores
Mobile-first	mapa ocupa 100% da largura e painel abre como Drawer/Sheet
Desktop	mapa e painel aparecem em duas colunas em breakpoint adequado
Mapa Brasil	cinco regiões renderizadas e identificáveis
Drill-down regional	somente Sudeste avança para estados
Mapa Sudeste	ES, MG, RJ e SP renderizados
Drill-down estadual	somente RJ avança para municípios
Mapa RJ	malha municipal visível
Dados municipais	quatro municípios cobertos recebem dados/heatmap
Sem cobertura	nunca e apresentado como zero casos
Breadcrumb	permite entender e retornar entre os níveis
Interação	mouse, toque e teclado possuem caminhos equivalentes
Filtros	doença, ano e mês alteram os dados exibidos
Join	geometria e dados ligados por código oficial, não por nome
Dados	mock e API obedecem aos mesmos tipos
Estados	loading, error, empty, no coverage e success existem
Acoplamento	nenhum componente visual chama <code>fetch</code> diretamente
Arquitetura	navegador conversa apenas com Java no modo real
Cartografia	GeoJSON local; nenhum PNG-grid como mecanismo principal
Entrega	uma única dashboard coerente e apresentável

22 Evolução posterior

Depois da demonstração, a sequência recomendada é:

1. finalizar persistência Gold do Python no PostgreSQL;
2. implementar repository/service/controller epidemiológico no Java;
3. adicionar OpenAPI e estabilizar DTOs;
4. integrar o front com a API Java real;
5. ampliar gradualmente cobertura geográfica e declarar explicitamente unidades completas;
6. implementar autenticação/autorização no Java e somente depois experiência de login no front;
7. expandir dimensões epidemiológicas com contrato SQL e API versionados.

23 Resumo executivo

A V1 do `nss-front-end` é uma dashboard cartográfica mobile-first. Seu fluxo principal é Brasil por regiões, drill-down para Sudeste, drill-down para Rio de Janeiro e visualização

dos quatro municipios atualmente cobertos pelo prototipo. O mapa e requisito central, nao opcional.

A cartografia deve ser vetorial, gerada por Python/**geobr** em desenvolvimento e versionada como GeoJSON. React Simple Maps renderiza as geometrias como SVG e permite interacao sem depender de mapas de ruas ou servicos externos. O estado geografico permanece separado dos filtros epidemiologicos, e os dados sao associados a geometria por codigos oficiais.

A interface distingue explicitamente *zero casos* de *sem cobertura*, evitando apresentar agregacoes incompletas como totais estaduais, regionais ou nacionais. Em telas pequenas, o mapa ocupa toda a largura e os filtros aparecem em Drawer; em telas maiores, mapa e painel ficam lado a lado com proporcao aproximada 70/30 ajustada por teste visual.

Essa abordagem mantem o escopo enxuto para a demonstracao e, ao mesmo tempo, cria uma base coerente para evoluir a cobertura geografica e conectar a API Java definitiva.